🎬 EasyMotion Toolkit — Full App Reverse Engineering & Development Guide

> IMPORTANT
>
> This is a complete breakdown of the **EasyMotion Toolkit** by Jamie Fenn — a DaVinci Resolve Studio script that automates the creation of **animated captions/subtitles** using **Fusion Text+** nodes. This guide covers the full app architecture, every UI feature, and a step-by-step plan to build your own version for **both macOS and Windows**.

1. App Identity & Overview

Property	Detail
App Name	EasyMotion Toolkit
Creator	Jamie Fenn (jamiefenn.com)
Platform	DaVinci Resolve Studio 21+
Type	DaVinci Resolve Script (launched via `Workspace > Scripts`)
UI Technology	Python + Qt/PySide2 UIManager (native Resolve script UI)
Core Function	Converts subtitles → animated Text+ Fusion clips with motion effects
Runs On	macOS & Windows (cross-platform Python)

2. Complete Feature Map (From Video Analysis)

2.1 App Window Structure

2.2 Tab 1: Workspace Tab (Main Editing)

This is the **primary workspace** where you style individual caption words:

Element	Function
Playhead / In/Out Toggle	Switch between "at playhead" mode and "in/out range" mode for selecting captions
Layout Icons (═ ═══ ≡)	3 text layout templates: single line, double line, triple line stacking
+ Text Button	Add a new Text+ clip manually to the timeline
Canvas Preview	Live black preview canvas showing styled text with zoom (100%, +/-)
Left Sidebar Tools	6 icon buttons for: Copy style, Duplicate, Position/Move, Resize, Target/Crosshair, Text editor (Aa)
Undo/Redo	↩️ ↪️ arrows at top-right of preview
Fullscreen Preview	⛶ button to expand preview
Refresh	🔄 circular arrow to reload state

Text Styling Panel (appears when selecting a word):

Control	Detail
Text Input Field	Edit the word text, with ⭐ (favorite) and ✕ (clear)
Font Picker	Dropdown showing font family (e.g., "Poppins") with 🔍 search
Font Weight	Dropdown (e.g., "Bold", "Black Italic")
Case Toggle	`aa` / `Aa` / `AA` buttons for case transformation
Color Swatch	Clickable color picker (shows current fill color)
Size Slider	Range slider with value (e.g., `0.1616`)
Tracking Slider	Letter spacing with value (e.g., `1`)
Opacity Slider	0–1 range
Effect Tabs	Gradient · Borders · Outline · Shadow · Glow · Blur
Outline Controls	Width slider, Color picker, "Turn off Outline" button
Position Section	Expandable position controls
Style Section	Save/Load style presets

2.3 Tab 2: Titles Tab (Style Gallery)

A **visual grid gallery** of saved title style presets:

Element	Function
Playhead / In/Out Toggle	Same as Workspace tab
Import Button	Import external style presets
Search (Q) / Sort (Se)	Filter and sort presets
+ Add from timeline	Capture current timeline title as a new preset
Style Grid	3-column grid of preset thumbnails with names
Preset Cards	Each shows: preview thumbnail, style name (e.g., "EM BOLD", "EM BOUNCY", "NOWY", "21", "helvetica"), variant count
Caption at playhead	Status showing current playhead caption
Edit in Auto →	Jump to Auto tab for the selected style

2.4 Tab 3: Auto Tab (Batch Automation)

The **most powerful feature —** bulk-generates all captions automatically:

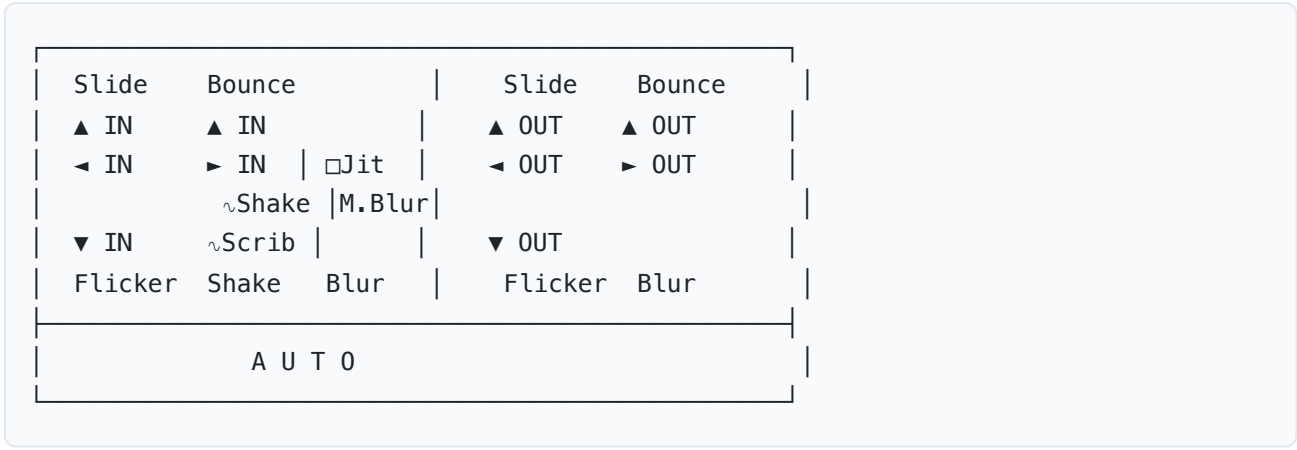
Element	Function
Style Selector	Shows current style with thumbnail + name (e.g., "21"), "Change >" button
Words per slide	Slider: "Fewer ← → More" (e.g., "up to 2 words", "up to 3 words")
Punctuation	Toggle: `Keep` / `Remove`
Animation	Toggle: `From style` / `Customize` / `Off`
Preview Section	Horizontal scrollable preview carousel showing all caption slides
Slide Counter	e.g., "6/12" with timecode range
Template Selector	`Template 1` (single line) · `Template 2` (double) · `Template 3` (triple)
Size Slider	Global size: `Size · all` with percentage
Look Toggle	`Standard` / `Style`
Timeline Scrubber	Mini playback scrubber with ▶ play button
Update caption	Update existing caption in-place
Create captions	🔴 Red button — generates all captions on timeline

Animation Customize Panel (expanded):

Control	Range
Slide	Off ← → On
Bounce	Off ← → In & Out
Directions	`Mixed` / `From style`
Flicker	Off ← → Every caption (with "Sometimes" option)
Shake IN	Off ← → Every caption
Blur IN	Off ← → Every caption
Done button	Save customization

2.5 Motion Effects Panel (Bottom Section - Always Visible)

This is the **animation control grid** at the bottom of the Workspace tab:



Left side = IN animations, Right side = OUT animations

Each effect button can be toggled ON (red/orange) or OFF. When active, the effect is applied to the selected caption's entrance or exit.

2.6 Convert Captions Feature

At the top header **— "Convert Captions"** button:

Element	Function
Subtitle range	e.g., "Subtitle 1 · 22" (source subtitle track)
Target type	"Toolkit Title" (convert to Text+ clips)
Look from style...	Apply a style preset during conversion
Remove punctuation	Checkbox toggle
Convert 22 · In/Out	🔴 Execute conversion for the range

2.7 Status Bar (Bottom)

Always shows contextual info:

- `"Done — 22 captions created on a new track."`
- `"4 words: Template 2 (⌘Z undoes)"`
- `"Selection and main words cleared."`
- `"Pasted (everything) into 1 words · ⌘Z undoes"`
- `"Clipboard: 'GUYS' · Poppins Black Italic"`
- `"Fonts loaded and measured · writes to Resolve confirmed"`

- `"SAMPLE · 1080x1920 · 0 captions at the playhead"`
- `"Inserted the In/Out words in NEW ONE · linked to 'MIGHT BE THE EASIEST' (AUTO)"`

3. Technical Architecture

3.1 How It Works Internally

3.2 Integration Type

This is a **DaVinci Resolve Script** (NOT a Workflow Integration Plugin). Key differences:

Property	Script (EasyMotion uses this)	Workflow Integration
UI Framework	Python + Qt UIManager	HTML/CSS/JS (Electron)
Location	`Workspace > Scripts`	`Workspace > Workflow Integrations`
Window Type	Native Qt window (separate)	Embedded panel inside Resolve
API Access	Full scripting API	Limited Workflow API
Requires	DaVinci Resolve Studio	DaVinci Resolve Studio

3.3 File Structure

```
EasyMotion Toolkit/
├── main.py                    # Entry point launched by Resolve
├── ui/
│   ├── main_window.py         # Main window with tab container
│   ├── workspace_tab.py       # Workspace tab UI + logic
│   ├── titles_tab.py          # Titles gallery tab
│   ├── auto_tab.py            # Auto generation tab
│   ├── motion_panel.py        # Bottom motion effects grid
│   ├── text_editor.py         # Text styling panel
│   ├── canvas_preview.py      # Live preview renderer
│   └── convert_dialog.py      # Convert Captions dialog
├── core/
│   ├── resolve_api.py         # DaVinci Resolve API wrapper
│   ├── timeline_manager.py    # Timeline manipulation
│   ├── subtitle_parser.py     # Read/parse subtitle tracks
│   ├── text_plus_builder.py   # Create & configure Text+ nodes
│   ├── animation_engine.py    # Keyframe animation logic
│   ├── style_manager.py       # Save/Load/Apply style presets
│   └── font_manager.py        # Font enumeration & measurement
├── data/
│   ├── styles/                # Saved style presets (JSON/YAML)
│   ├── templates/             # Fusion .setting template files
│   └── config.json            # App configuration
└── resources/
    ├── icons/                 # UI icons
    └── fonts/                 # Bundled fonts (if any)
```

4. Core Technologies Required

4.1 For Building Your Own Version

Technology	Purpose	Platform
Python 3.6+	Main scripting language	Both
DaVinciResolveScript	Resolve API module	Both
PySide2 / PySide6	Qt UI framework (or Resolve's built-in UIManager)	Both
Fusion Scripting	Text+ node manipulation	Both
JSON	Style preset storage	Both
PIL/Pillow (optional)	Font measurement & preview rendering	Both

4.2 DaVinci Resolve API Key Methods

```
python
# Connection
resolve = dvr.scriptapp("Resolve")
project = resolve.GetProjectManager().GetCurrentProject()
timeline = project.GetCurrentTimeline()
media_pool = project.GetMediaPool()

# Subtitle Operations
subtitle_track = timeline.GetItemListInTrack("subtitle", 1)

# Timeline Operations
timeline.GetTrackCount("video")
timeline.AddTrack("video")
media_pool.AppendToTimeline([{...}])

# Fusion Text+ Operations
item = timeline.GetItemListInTrack("video", track_idx)[clip_idx]
comp = item.GetFusionCompByIndex(1)
text_plus = comp.FindTool("Template")  # or specific tool name

# Text+ Properties
text_plus.SetInput("StyledText", "HELLO WORLD")
text_plus.SetInput("Font", "Poppins")
text_plus.SetInput("Style", "Bold")
text_plus.SetInput("Size", 0.1)
text_plus.SetInput("Red1", 1.0)   # Color channels
text_plus.SetInput("Green1", 0.0)
text_plus.SetInput("Blue1", 0.0)
text_plus.SetInput("Tracking", 1.05)
text_plus.SetInput("Opacity", 1.0)
```

5. Step-by-Step Development Plan

> NOTE
>
> This is the implementation roadmap for building your own "MotionResolve" toolkit. Each phase builds on the previous one.

Phase 1: Foundation & Resolve Connection (Week 1)

Goal: Get a Python script that launches from Resolve's Scripts menu and connects to the API.

1. **Set up the development environment**
    - Install Python 3.6+ (match Resolve's bundled Python version)
    - Configure `RESOLVE_SCRIPT_API` and `PYTHONPATH` environment variables
    - **macOS:** `export RESOLVE_SCRIPT_API="/Library/Application Support/Blackmagic Design/DaVinci Resolve/Developer/Scripting/"`
    - **Windows:** `set RESOLVE_SCRIPT_API="C:\ProgramData\Blackmagic Design\DaVinci Resolve\Support\Developer\Scripting\"`
2. **Create the entry point script**

```
python
#!/usr/bin/env python3
# main.py — Entry point for Motion Resolve
import sys, os

# Add Resolve script modules to path
script_module = os.environ.get("RESOLVE_SCRIPT_API", "")
if script_module:
    sys.path.append(os.path.join(script_module, "Modules"))

import DaVinciResolveScript as dvr

def get_resolve():
    resolve = dvr.scriptapp("Resolve")
    if not resolve:
        print("ERROR: Cannot connect to DaVinci Resolve")
        sys.exit(1)
    return resolve
```

3. **Install the script in Resolve's Scripts directory**
    - **macOS:** `~/Library/Application Support/Blackmagic Design/DaVinci Resolve/Fusion/Scripts/Comp/`
    - **Windows:** `%APPDATA%\Blackmagic Design\DaVinci Resolve\Support\Fusion\Scripts\Comp\`

Phase 2: UI Shell with Tabs (Week 2)

Goal: Build the main window with 3 tabs using Resolve's UIManager (Qt).

```
python
# ui/main_window.py
def create_main_window(resolve):
    ui = fusion.UIManager
    disp = bmd.UIDispatcher(ui)

    window = disp.AddWindow({
        "ID": "MotionResolveToolkit",
        "WindowTitle": "Motion Resolve",
        "Geometry": [100, 100, 400, 800],
    }, [
        ui.VGroup([
            # Header
            ui.HGroup([
                ui.Label({"Text": "motionresolve", "Weight": 1}),
                ui.Button({"ID": "ConvertBtn", "Text": "Convert Captions"}),
                ui.Button({"ID": "SettingsBtn", "Text": "..."}),
            ]),
            # Tabs
            ui.TabBar({"ID": "MainTabs", "Weight": 0}),
            ui.Stack({"ID": "TabStack", "Weight": 1}, [
                # Tab 1: Workspace
                ui.VGroup({"ID": "WorkspaceTab"}, [...]),
                # Tab 2: Titles
                ui.VGroup({"ID": "TitlesTab"}, [...]),
                # Tab 3: Auto
                ui.VGroup({"ID": "AutoTab"}, [...]),
            ]),
            # Motion Effects Panel (always visible)
            ui.VGroup({"ID": "MotionPanel"}, [...]),
            # Status Bar
            ui.Label({"ID": "StatusBar", "Text": "Ready"}),
        ])
    ])
    return window, disp
```

Phase 3: Subtitle Reading & Parsing (Week 3)

Goal: Read the subtitle track from the timeline and extract word-level timing.

```
python
# core/subtitle_parser.py
class SubtitleParser:
    def __init__(self, timeline):
        self.timeline = timeline

    def get_subtitles(self):
        """Read all subtitle items from the subtitle track."""
        subs = []
        track_count = self.timeline.GetTrackCount("subtitle")
        for track_idx in range(1, track_count + 1):
            items = self.timeline.GetItemListInTrack("subtitle", track_idx)
            if items:
                for item in items:
                    subs.append({
                        "text": item.GetName(),
                        "start": item.GetStart(),
                        "end": item.GetEnd(),
                        "duration": item.GetDuration(),
                    })
        return subs

    def split_into_words(self, subs, words_per_slide=2):
        """Split subtitle phrases into individual word groups."""
        word_groups = []
        for sub in subs:
            words = sub["text"].split()
            total_frames = sub["duration"]
            frames_per_word = total_frames / len(words)

            for i in range(0, len(words), words_per_slide):
                group = words[i:i + words_per_slide]
                group_start = sub["start"] + int(i * frames_per_word)
                group_end = sub["start"] + int((i + len(group)) * frames_per_word)
                word_groups.append({
                    "text": " ".join(group),
                    "words": group,
                    "start": group_start,
                    "end": group_end,
                })
        return word_groups
```

Phase 4: Text+ Creation Engine (Week 4)

Goal: Programmatically create Text+ Fusion clips on the timeline.

```
python
# core/text_plus_builder.py
class TextPlusBuilder:
    def __init__(self, resolve, project, timeline):
        self.resolve = resolve
        self.project = project
        self.timeline = timeline
        self.media_pool = project.GetMediaPool()

    def create_text_clip(self, text, start_frame, end_frame, track_index,
                         style=None):
        """Create a Fusion Text+ clip on the timeline."""
        # Step 1: Add a Fusion Generator to the timeline
        # Use the media pool to append a generator
        duration = end_frame - start_frame

        # Create timeline item via Fusion composition
        # This approach creates a Fusion clip container
        self.timeline.SetCurrentTimecode(
            self._frames_to_timecode(start_frame)
        )

        # Insert a Text+ generator
        # Access via Fusion composition
        fusion = self.resolve.Fusion()

        # Alternative: Use template .setting files
        # Load a pre-made Text+ template and modify it
        pass

    def apply_style(self, text_plus_tool, style):
        """Apply a saved style preset to a Text+ node."""
        if not style:
            return
        text_plus_tool.SetInput("Font", style.get("font", "Arial"))
        text_plus_tool.SetInput("Style", style.get("weight", "Bold"))
        text_plus_tool.SetInput("Size", style.get("size", 0.1))
        text_plus_tool.SetInput("Tracking", style.get("tracking", 1.0))
        text_plus_tool.SetInput("Opacity", style.get("opacity", 1.0))

        # Colors
        if "color" in style:
            r, g, b = style["color"]
            text_plus_tool.SetInput("Red1", r)
            text_plus_tool.SetInput("Green1", g)
```

```
            text_plus_tool.SetInput("Blue1", b)

        # Outline
        if style.get("outline_enabled"):
            text_plus_tool.SetInput("ElementShape1", 1)  # Enable outline
            text_plus_tool.SetInput("Thickness1", style.get("outline_width", 0.05))

        # Shadow
        if style.get("shadow_enabled"):
            text_plus_tool.SetInput("ShadowEnabled", 1)
```

Phase 5: Animation Engine (Week 5-6)

Goal: Implement all motion effects (Slide, Bounce, Shake, Flicker, Blur, Scribble, Jitter).

```
python
# core/animation_engine.py
class AnimationEngine:
    """
    Applies keyframe animations to Text+ Fusion nodes.
    All animations work by setting keyframes on Text+ transform properties.
    """

    def apply_slide_in(self, comp, text_tool, direction="up", duration_frames=5):
        """Slide text in from a direction."""
        start = comp.RenderStart
        offset_map = {
            "up": (0, -0.3),
            "down": (0, 0.3),
            "left": (-0.5, 0),
            "right": (0.5, 0),
        }
        dx, dy = offset_map.get(direction, (0, -0.3))

        # Get current center position
        cx = text_tool.GetInput("Center")[1]
        cy = text_tool.GetInput("Center")[2]

        # Keyframe: off-screen at start, on-screen after duration
        text_tool.Center = comp.BezierSpline({})
        text_tool.Center[start] = {cx + dx, cy + dy}
        text_tool.Center[start + duration_frames] = {cx, cy}

    def apply_bounce_in(self, comp, text_tool, duration_frames=8):
        """Bouncy scale-up entrance."""
        start = comp.RenderStart
        text_tool.Size = comp.BezierSpline({})
        base_size = text_tool.GetInput("Size")
        text_tool.Size[start] = 0.0
        text_tool.Size[start + int(duration_frames * 0.6)] = base_size * 1.2
        text_tool.Size[start + duration_frames] = base_size

    def apply_shake(self, comp, text_tool, intensity=0.01, duration_frames=3):
        """Quick shake effect on entrance."""
        start = comp.RenderStart
        cx = text_tool.GetInput("Center")[1]
        cy = text_tool.GetInput("Center")[2]

        import random
        for i in range(duration_frames):
```

```
            jx = cx + random.uniform(-intensity, intensity)
            jy = cy + random.uniform(-intensity, intensity)
            text_tool.Center[start + i] = {jx, jy}
        text_tool.Center[start + duration_frames] = {cx, cy}

    def apply_flicker(self, comp, text_tool, duration_frames=4):
        """Opacity flicker effect."""
        start = comp.RenderStart
        text_tool.Opacity = comp.BezierSpline({})
        text_tool.Opacity[start] = 0.0
        text_tool.Opacity[start + 1] = 1.0
        text_tool.Opacity[start + 2] = 0.3
        text_tool.Opacity[start + 3] = 1.0
        text_tool.Opacity[start + duration_frames] = 1.0

    def apply_blur_in(self, comp, text_tool, duration_frames=5):
        """Blur-to-sharp entrance."""
        # Add a Blur node after Text+ and animate its size
        blur = comp.AddTool("Blur", -1, -1)
        blur.SetInput("XBlurSize", 10.0)
        blur.XBlurSize = comp.BezierSpline({})
        blur.XBlurSize[comp.RenderStart] = 10.0
        blur.XBlurSize[comp.RenderStart + duration_frames] = 0.0

    def apply_motion_blur(self, comp, text_tool, duration_frames=5):
        """Directional motion blur entrance."""
        dblur = comp.AddTool("DirectionalBlur", -1, -1)
        dblur.Length = comp.BezierSpline({})
        dblur.Length[comp.RenderStart] = 0.15
        dblur.Length[comp.RenderStart + duration_frames] = 0.0

    def apply_auto(self, comp, text_tool, style_config):
        """Apply all animations defined in a style config."""
        if style_config.get("slide_in"):
            self.apply_slide_in(comp, text_tool,
                                style_config.get("slide_direction", "up"))
        if style_config.get("bounce_in"):
            self.apply_bounce_in(comp, text_tool)
        if style_config.get("shake_in"):
            self.apply_shake(comp, text_tool)
        if style_config.get("flicker"):
            self.apply_flicker(comp, text_tool)
        if style_config.get("blur_in"):
            self.apply_blur_in(comp, text_tool)
```

Phase 6: Style Management (Week 7)

Goal: Save, load, and apply text style presets (the Titles tab gallery).

```
python
# core/style_manager.py
import json
import os

class StyleManager:
    def __init__(self, styles_dir):
        self.styles_dir = styles_dir
        os.makedirs(styles_dir, exist_ok=True)
        self.styles = self._load_all()

    def _load_all(self):
        styles = {}
        for f in os.listdir(self.styles_dir):
            if f.endswith(".json"):
                with open(os.path.join(self.styles_dir, f)) as fp:
                    style = json.load(fp)
                    styles[style["name"]] = style
        return styles

    def save_style(self, name, config):
        """Save a style preset."""
        config["name"] = name
        path = os.path.join(self.styles_dir, f"{name}.json")
        with open(path, "w") as fp:
            json.dump(config, fp, indent=2)
        self.styles[name] = config

    def capture_from_timeline(self, text_plus_tool, name):
        """Capture the current Text+ node's style as a preset."""
        config = {
            "name": name,
            "font": text_plus_tool.GetInput("Font"),
            "weight": text_plus_tool.GetInput("Style"),
            "size": text_plus_tool.GetInput("Size"),
            "tracking": text_plus_tool.GetInput("Tracking"),
            "color": [
                text_plus_tool.GetInput("Red1"),
                text_plus_tool.GetInput("Green1"),
                text_plus_tool.GetInput("Blue1"),
            ],
            "opacity": text_plus_tool.GetInput("Opacity"),
            "outline_enabled": bool(text_plus_tool.GetInput("ElementShape1")),
            "outline_width": text_plus_tool.GetInput("Thickness1"),
            "animation": {
                "slide_in": True,
```

```
                "slide_direction": "up",
                "bounce_in": False,
                "flicker": False,
                "shake_in": False,
                "blur_in": False,
            }
        }
        self.save_style(name, config)
        return config
```

Phase 7: Canvas Preview (Week 8)

Goal: Render a live preview of the styled text in the app window.

```
python
# core/preview_renderer.py
class PreviewRenderer:
    """
    Renders a preview of styled text using Pillow or Qt's QPainter.
    This shows the user what their caption will look like without
    needing to play the timeline.
    """

    def render_preview(self, text, style, width=400, height=300):
        """Generate a preview image of the styled text."""
        from PIL import Image, ImageDraw, ImageFont

        img = Image.new("RGB", (width, height), (0, 0, 0))
        draw = ImageDraw.Draw(img)

        # Load font
        try:
            font = ImageFont.truetype(style.get("font_path", ""),
                                      int(style.get("size", 40) * 400))
        except:
            font = ImageFont.load_default()

        # Calculate text position (centered)
        bbox = draw.textbbox((0, 0), text, font=font)
        tw, th = bbox[2] - bbox[0], bbox[3] - bbox[1]
        x = (width - tw) // 2
        y = (height - th) // 2

        # Draw outline if enabled
        if style.get("outline_enabled"):
            outline_color = tuple(int(c * 255) for c in style.get("outline_color",
[0,0,0]))
            for dx in range(-2, 3):
                for dy in range(-2, 3):
                    draw.text((x+dx, y+dy), text, font=font, fill=outline_color)

        # Draw main text
        color = tuple(int(c * 255) for c in style.get("color", [1,1,1]))
        draw.text((x, y), text, font=font, fill=color)

        return img
```

Phase 8: Auto Tab — Batch Generation (Week 9-10)

Goal: The full auto-generation pipeline.

```
python
# core/auto_generator.py
class AutoGenerator:
    """
    The core automation engine. Takes subtitle data + style preset
    and generates all caption clips on the timeline automatically.
    """

    def __init__(self, resolve, project, timeline, style_manager,
                 animation_engine, text_builder):
        self.resolve = resolve
        self.project = project
        self.timeline = timeline
        self.style_mgr = style_manager
        self.anim_engine = animation_engine
        self.text_builder = text_builder

    def generate_all(self, subtitles, style_name, words_per_slide=2,
                     remove_punctuation=True, template=1):
        """
        Main entry point: generate all animated captions.

        1. Parse subtitles into word groups
        2. Create a new video track
        3. For each word group, create a Text+ clip
        4. Apply style + animations
        """
        style = self.style_mgr.styles.get(style_name)
        if not style:
            raise ValueError(f"Style '{style_name}' not found")

        # Parse into word groups
        parser = SubtitleParser(self.timeline)
        word_groups = parser.split_into_words(subtitles, words_per_slide)

        # Clean punctuation if requested
        if remove_punctuation:
            import re
            for wg in word_groups:
                wg["text"] = re.sub(r'[^\w\s]', '', wg["text"])
                wg["words"] = [re.sub(r'[^\w\s]', '', w) for w in wg["words"]]

        # Create new video track(s) based on template
        track_count = template  # Template 1=1 track, 2=2, 3=3
        new_tracks = []
```

```
        for i in range(track_count):
            self.timeline.AddTrack("video")
            idx = self.timeline.GetTrackCount("video")
            new_tracks.append(idx)

        # Generate each caption
        total = len(word_groups)
        for i, wg in enumerate(word_groups):
            # Distribute across tracks based on template
            target_track = new_tracks[i % track_count]

            # Create Text+ clip
            clip = self.text_builder.create_text_clip(
                text=wg["text"].upper(),
                start_frame=wg["start"],
                end_frame=wg["end"],
                track_index=target_track,
                style=style,
            )

            # Apply animations
            if style.get("animation"):
                comp = clip.GetFusionCompByIndex(1)
                if comp:
                    text_tool = comp.FindTool("Template")
                    if text_tool:
                        self.anim_engine.apply_auto(comp, text_tool,
                                                    style["animation"])

            # Report progress
            progress = int((i + 1) / total * 100)
            print(f"Progress: {progress}% — Created '{wg['text']}'")

        return total
```

6. Cross-Platform Installation Guide

macOS Installation Paths

```
bash
# Script location (appears in Workspace > Scripts)
~/Library/Application Support/Blackmagic Design/DaVinci
Resolve/Fusion/Scripts/Comp/MotionResolve/

# Style presets storage
~/Library/Application Support/Blackmagic Design/DaVinci
Resolve/Fusion/Scripts/Comp/MotionResolve/data/styles/

# Fusion templates
~/Library/Application Support/Blackmagic Design/DaVinci
Resolve/Fusion/Templates/Edit/Titles/MotionResolve/

# Environment variables (add to ~/.zshrc)
export RESOLVE_SCRIPT_API="/Library/Application Support/Blackmagic Design/DaVinci
Resolve/Developer/Scripting/"
export RESOLVE_SCRIPT_LIB="/Applications/DaVinci Resolve/DaVinci
Resolve.app/Contents/Libraries/Fusion/fusionscript.so"
export PYTHONPATH="$RESOLVE_SCRIPT_API/Modules:$PYTHONPATH"
```

Windows Installation Paths

```
powershell
# Script location
%APPDATA%\Blackmagic Design\DaVinci
Resolve\Support\Fusion\Scripts\Comp\MotionResolve\

# Style presets storage
%APPDATA%\Blackmagic Design\DaVinci
Resolve\Support\Fusion\Scripts\Comp\MotionResolve\data\styles\

# Fusion templates
%APPDATA%\Blackmagic Design\DaVinci
Resolve\Support\Fusion\Templates\Edit\Titles\MotionResolve\

# Environment variables (System Properties > Environment Variables)
RESOLVE_SCRIPT_API=C:\ProgramData\Blackmagic Design\DaVinci
Resolve\Support\Developer\Scripting\
RESOLVE_SCRIPT_LIB=C:\Program Files\Blackmagic Design\DaVinci
Resolve\fusionscript.dll
PYTHONPATH=%RESOLVE_SCRIPT_API%\Modules
```

Cross-Platform Path Helper

```
python
# core/paths.py
import os, platform

def get_resolve_paths():
    """Return platform-specific paths."""
    system = platform.system()

    if system == "Darwin":  # macOS
        base = os.path.expanduser(
            "~/Library/Application Support/Blackmagic Design/DaVinci Resolve"
        )
        return {
            "scripts": os.path.join(base, "Fusion/Scripts/Comp"),
            "templates": os.path.join(base, "Fusion/Templates/Edit/Titles"),
            "api": "/Library/Application Support/Blackmagic Design/DaVinci
Resolve/Developer/Scripting/",
            "lib": "/Applications/DaVinci Resolve/DaVinci
Resolve.app/Contents/Libraries/Fusion/fusionscript.so",
        }

    elif system == "Windows":
        base = os.path.join(os.environ["APPDATA"],
                            "Blackmagic Design/DaVinci Resolve/Support")
        return {
            "scripts": os.path.join(base, "Fusion/Scripts/Comp"),
            "templates": os.path.join(base, "Fusion/Templates/Edit/Titles"),
            "api": "C:/ProgramData/Blackmagic Design/DaVinci
Resolve/Support/Developer/Scripting/",
            "lib": "C:/Program Files/Blackmagic Design/DaVinci
Resolve/fusionscript.dll",
        }

    raise RuntimeError(f"Unsupported platform: {system}")
```

7. Development Roadmap Summary

> TIP

> **Start with Phase 1 and 3 first —** getting the Resolve API connection and subtitle parser working is the hardest part. Once you can read subtitles and create Text+ clips programmatically, everything else is UI and polish.

8. Key Technical Challenges & Solutions

Challenge	Solution
Resolve API has no direct subtitle creation	Convert subtitles to Text+ Fusion clips on video tracks
Font measurement for preview	Use Pillow or system font APIs to measure text bounds
Cross-platform font paths	Use `matplotlib.font_manager` or OS-specific font directories
Animation timing precision	Work in frames (not seconds) matching timeline FPS
Undo support	Resolve scripts can use `timeline.Undo()` — group operations
Style gallery thumbnails	Pre-render preview images with Pillow and cache as PNGs
Real-time preview sync	Poll playhead position and update canvas accordingly
Template (.setting) files	Pre-build Fusion node trees and load them as templates

> WARNING
>
> **DaVinci Resolve Studio is REQUIRED —** The free version does not expose the scripting API. Your users must have Studio to use this tool.

9. Next Steps

When you're ready to start building, I recommend using the `/plan` command to create a detailed implementation plan for Phase 1-2, and then we can start coding the actual app together.

Alternatively, use `/boost` for a deep-dive development session where I'll build the complete foundation with you.